Forward Deployed Engineer - Task Details

Graph-Based Data Modeling and Query System

Overview

In real-world business systems, data is spread across multiple tables : orders, deliveries, invoices, and payments, without a clear way to trace how they connect.

In this assignment, you will unify this fragmented data into a graph and build a system that allows users to explore and query these relationships using natural language.

What You Are Building

You are building a **context graph system with an LLM-powered query interface**. Below is a sample interface for reference:

At a high level:

- **graph of interconnected entities**
- This graph is **visualized in a UI**
- A **chat interface sits alongside the graph**
- The user asks questions in natural language
- The system translates those questions into **structured queries (such as SQL) dynamically**
- The system executes those queries and returns **data-backed answers in natural language**

This is not a static Q&A system. The LLM should interpret user queries, generate structured queries dynamically, and return data-backed answers.

Dataset

First, please download this dataset:

https://drive.google.com/drive/folders/1UqaLbFaveV-3MEuiUrzKydhKmkeC1iAL

The dataset includes entities such as:

Core Flow

- Orders
- Deliveries
- Invoices
- Payments

Supporting Entities

- Customers
- Products
- Address

You are free to preprocess, normalize, or restructure the dataset as required.

Functional Requirements

1. Graph Construction

Ingest the dataset and construct a graph representation.

You must define:

- Nodes representing business entities
- Edges representing relationships between entities

Examples of relationships:

- Purchase Order → Purchase Order Item
- Delivery → Plant
- Purchase Order Item → Material
- Customer → Delivery

The focus is on how you model the system, not just loading data.

2. Graph Visualization

Build an interface that allows users to explore the graph.

The interface should support:

- Expanding nodes
- Inspecting node metadata
- Viewing relationships between entities

A simple and clean implementation is sufficient.

You may use any visualization library of your choice.

3. Conversational Query Interface

Build a chat interface that allows users to query the system.

The system should:

- Accept natural language queries
- Translate queries into structured operations on the graph or underlying data
- Return accurate and relevant responses

The responses must be grounded in the dataset and not generated without data backing.

4. Example Queries

Your system should be capable of answering questions such as:

a. Which products are associated with the highest number of billing documents?
b. Trace the full flow of a given billing document (Sales Order → Delivery → Billing → Journal Entry)
c. Identify sales orders that have broken or incomplete flows (e.g. delivered but not billed, billed without delivery)

You are encouraged to go beyond these examples and explore additional meaningful queries based on your understanding of the dataset.

5. Guardrails

The system must restrict queries to the dataset and domain.

It should appropriately handle or reject unrelated prompts such as:

- General knowledge questions
- Creative writing requests
- Irrelevant topics

Example response:

"This system is designed to answer questions related to the provided dataset only."

This is an important evaluation criterion.

Optional Extensions (Bonus)

- Natural language to SQL or graph query translation
- Highlighting nodes referenced in responses
- Semantic or hybrid search over entities
- Streaming responses from the LLM
- Conversation memory
- Graph clustering or advanced graph analysis

Depth in one or two functionalities is preferred over implementing many superficially.

LLM APIs: Use Free Tiers

You don't need to spend money on this.

Several providers offer free access with reasonable limits.

Provider	Link
Google Gemini	https://ai.google.dev
Groq	https://console.groq.com
OpenRouter	https://openrouter.ai
HuggingFace	https://huggingface.co/inference-api
Cohere	https://cohere.com

Submission Requirements

In order to record your submission:

- A **working demo link**
- A **public GitHub repository**
- A **README** explaining architecture decisions, database choice, LLM prompting strategy, and guardrails
- **AI coding session logs** from tools such as Cursor, Claude Code, Copilot, etc.
- The UI can be simple.

Share Your AI Coding Sessions

We expect candidates to actively use AI tools as part of this assignment.

We're interested in understanding **how you work with AI**, not just the final output.

If you're using tools such as:

- Cursor
- Claude Code
- GitHub Copilot
- Windsurf
- Continue.dev

Please include your **session logs or transcripts**.

Examples:

Cursor → Export your **Composer / chat history / Export Transcript** as markdown

Claude Code → Include transcripts from: ~/.claude/projects/

Other tools → Provide any logs in a markdown / .txt format

*Note: If you're using multiple tools, **provide the transcripts for each** of them and bundle them in to a .ZIP file*

We're evaluating:

- prompt quality
- debugging workflow
- iteration patterns

Evaluation Criteria

Area	What We Are Evaluating
Code quality and architecture	Structure, readability, and maintainability
Graph modelling	Quality and clarity of entities and relationships
Database / storage choice	Architectural decisions and tradeoffs
LLM integration and prompting	How natural language is translated into useful queries
Guardrails	Ability to restrict misuse and off-topic prompts